

# How I AI

## A Self-contained French Study Dashboard (one-page summary)

*This is a brief, at-a-glance summary. The full workflow and each module's features are more complex — the complete version is in [How-I-AI.md](#).*

**Name:** XU Shiqi

**Student number:** 3036761155

**AI tool:** AI coding assistant in VS Code

**Repository:** [https://github.com/1557302476-alt/DELFB2\\_preparation](https://github.com/1557302476-alt/DELFB2_preparation)

**Live page:** [https://1557302476-alt.github.io/DELFB2\\_preparation/](https://1557302476-alt.github.io/DELFB2_preparation/)

### 1. What I use AI for

I am preparing for the DELF B2 French exam and wanted a study **tool** that trains my reading, writing, listening and speaking comprehensively — not a spreadsheet. So I asked an AI coding assistant in VS Code to build **one self-contained web page** with six modules: a spaced-repetition plan, a home dashboard, a vocabulary self-test, a daily reading/listening drill built from fresh French news, a word-lookup bar with a personal wordbook, and auto-generated questions. I don't program: my job is to give a precise spec and check every output; the assistant writes and revises the code.

### 2. How you can do it too

Every module came from **one loop**, repeated six times: *write a precise spec* → *let the assistant generate the code* → *ask for a small test* → *fix what the test exposes*. My reusable "master prompt" sets the rules up front:

You are my coding assistant inside VS Code. Work only inside this folder.

Goal: <one-sentence goal>. Source material: <files in this folder>.

Hard rules:

1. Deliver ONE self-contained .html file (all CSS/JS inline, no CDN) so I can open it by double-click and publish it as-is.
2. Keep the source data in separate plain .py data files.
3. Provide a build script; after each change, run it and report the output size.
4. For every interactive feature, also write a tiny headless test and show me the pass/fail output.
5. Never invent data — if a source can't be fetched, leave it empty and say so.

Show me a short plan first; wait for my OK before writing code.

Then the loop: give it the raw material (two vocabulary PDFs → `vocab_data.py`, `vocab_new.py`) → ask for the schedule model → ask for the generator (`build_dashboard.py`, which turns the data into one `.html`) → run it in the terminal → open and actually use the page → iterate by reporting what I see. The

daily news part is automated with a small `fetch_news.py` run by a `launchd` task at 8:17am, plus a manual "update" button as a fallback.

### 3. One example

---

**My input.** *"When I select a French word in the article, a bottom bar should show its part of speech and Chinese meaning, and 加入生词本 should save all three together."*

**AI output.** A text-selection listener, a fixed lookup bar, and a save path that stores `{word, meaning, pos}` — e.g. `flambée · n.f./v. · 暴涨`.

**My check.** A headless test looked up six words and exposed a real bug: only 3/6 returned a meaning.

```
dérisoire -> 可笑 | inédit -> 新奇 | prévisible -> 可预见的  
surprenant -> (empty) | flambée -> (empty) | chômage -> (empty)    # 3 / 6
```

### 4. What went wrong, and what I changed

---

The empty results looked random, until I repeated the same six lookups 2 seconds apart and got 6/6 — so "empty" meant *rate-limited*, not *not found*. I fixed it by spacing out the requests, adding one automatic retry, and only caching a result that actually returned a meaning. **Lesson: separate a genuine miss from a transient failure.** (Two more fixes are in the full write-up: a CSS name collision that collapsed the self-test list, and a daily update that quietly pointed at a folder I had moved.)

---

Full version with screenshots: `How-I-AI.md`. All code is in the repository above.